System Design: the Northstar demo of the Forma company app platform, the demo's edition

The same design told act by act, for a YC partner judging the recording it plans and for the engineer who builds it: what each act shows, what makes it real, what is staged and labeled, and what to check first.

Author	Victor Villacis
Date	2026-10-23
Status	Proposed
Edition	demo, derived from the record, `design.md`
Read first	The partner: nothing. The engineer: `design.md`, then the platform record, `tasks/forma-generative-app-platform-v2/design.md`, for the mechanisms

Contents

Summary in plain words

What is real, what is labeled, and how the recording is cut

The system in two pictures

1. Act 1, entering the workspace

2. Act 2, bringing evidence

3. Act 3, forming and transforming

4. Act 4, publishing and running

5. Act 5, sharing and reusing

6. Act 6, a different app

7. What the demo is judged on, and what it does not prove

    What it is judged on, ranked

    What it does not prove

    The eight reductions and their seams

8. How it grows into the platform, and what we do not know yet

    How it grows

    What we considered instead

    The one-way doors

    What we do not know yet, and what to do first

9. Security and operations in brief, and the accepted risks

10. For the engineer

    Does it fit?

    Security in detail

    Operations in detail

    The cost

11. The map to the record

> This edition summarises the record, `design.md`, and adds nothing to it. The partner's twenty minutes run through chapter 9; chapter 10 is the engineer's, and chapter 11 maps each chapter to the record. Section numbers in parentheses point into the record for the engineer; the partner can skip them.

Summary in plain words

The Northstar demo is a recording for YC, answering its request A Cloud for Small Software: small software as easy to share with colleagues as a Google Doc. It shows the Forma company app platform doing what a platform must: two different apps formed and published through one path on one build, the first reused by a colleague. Northstar is fictional, and every row of data is synthetic. Maya imports two small files and asks in words for an investigation app. A model proposes it as a typed app definition, data the platform checks rather than code it trusts. Sarah, the reviewer, approves it; Jordan, a colleague, reopens Maya's work and starts his own on a file he imports. Then Maya forms Spend Review, a finance queue with a decision form, the same way, with no deploy between.

A recording of two fixed dashboards would prove none of this. So the demo runs the platform's own code and ships its proof with the video in a *packet*: the image digest of what ran, both app definitions, the receipts and the rehearsal log. A *receipt* is the server's record of exactly which sources, revisions, parameters and policy produced a number. The *canary*, a second synthetic organization, replays every call with Northstar's identifiers and must get nothing back; one replay is on camera.

The demo optimises for **generality** and **isolation**: one build digest, the fingerprint of the one software image, across all six acts, and nothing of Northstar reaching the canary. It pays in scale, availability, generation latency and cost: one process on one host, no standby, a restart that is a scene, and about $170 a month, hosting and tokens together. Its eight *reductions* are shortcuts that each name a *seam*, the platform mechanism that replaces it. A *take* is one recording of an act or of the whole run; a *rehearsal* is a timed whole run of about thirty minutes, about $17 each at ten a month.

The honest part: the status is Proposed. Built are Forma 3.0.0, the framework, at commit `2006423b`, on which the platform's review rendered a prototype canvas of four component kinds, and fAuth, Forma's own sign-in service, at commit `b750ca53`. The rest, the platform's four parts and the demo's tools, is designed, not built. A spike, the measurement before any scene, runs first, and no rehearsal starts before it passes.

What is real, what is labeled, and how the recording is cut

The question. What in the recording is working software, what is staged, and is it one take?

The answer. Every scene runs the platform's code on one build, every number with a receipt. Each staged step is captioned on screen as it happens, and the recording joins one kept take per act.

What is labeled. Every seeded, simulated or manual step is captioned on screen as it happens, logged, and left out of the measures a take is accepted on: its times, sample sizes and receipt checks. *Seeded*: each take's starting state, and the spike's fixture proposal, which never appears in a take. *Simulated*: faults the *rehearsal script* makes at the client, the server's answers real: act 4's late answer from the previous filter; act 6's dropped response, its stale and reused-key requests and its late Finance answer. *Manual*: act 4's restart, by the operator.

How the recording is cut. Before each take the *reset*, an operator tool, restores a saved state: `s0`, as provisioned, before act 1, and `s2` to `s5` after acts 2 to 5. There is no `s1`, since act 1 changes nothing a take restores. The recording joins one kept take per act, each starting from the state the last ended in, so act 4 approves what act 3 formed on camera. Retaking act N retakes the acts after it.

The partner's first questions.

- *One take or many?* One kept take per act; the rehearsal log counts every discarded take.
- *The model is live: why the same numbers every take?* They are queries over the fixtures; the model's arrangement of the views may vary.
- *What in act 3 came from the model?* Three views come from Northstar's search index over the catalog's metadata and curated examples, and one rule is adapted; the rest comes from one model call, and the two patches from controls.
- *What if I type my own request?* Inside the platform's closed vocabulary, its fixed list of component kinds and operations, it takes the same path; outside it, *admission*, the platform's check of every proposal, refuses it whole and names the missing kind. Each rehearsal films one labeled unscripted take of exactly this: a third app named by someone outside the team, filed as admitted or as refused.
- *Who is at the keyboard?* The operator, the founder, on one machine: Maya, Sarah and Jordan are three accounts in three Chrome profiles, brought forward one at a time by the rehearsal script, which adds no code to the product. Sarah's approval is a different account's, which is what Northstar's policy checks.

The system in two pictures

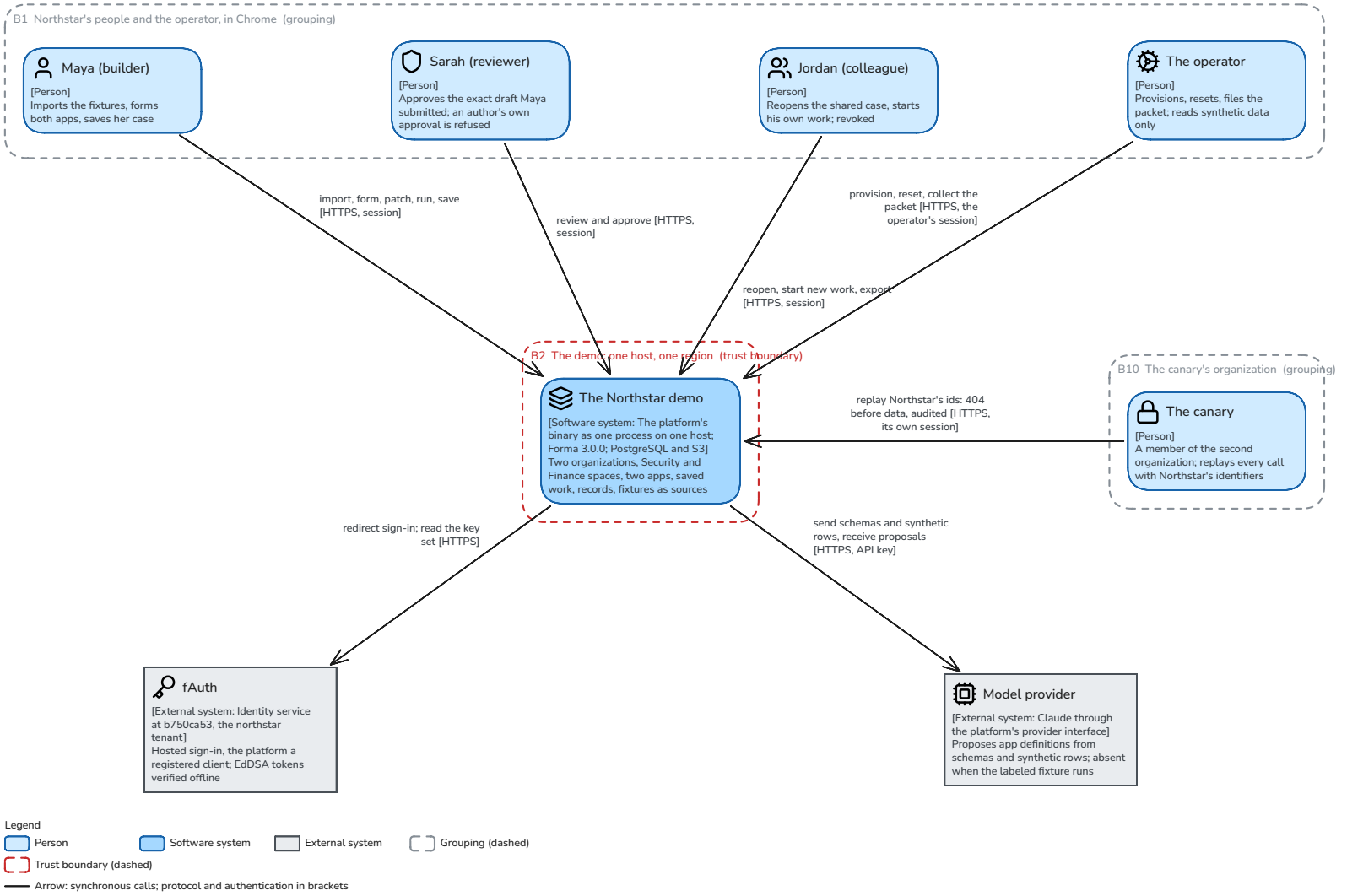
The question. What are the parts, and where does each act run?

The answer. The platform's four parts run as one program on one host over PostgreSQL and object storage, and one of them, the workspace server, decides what every request may do. fAuth sees who signs in, never data; the model provider sees schemas and synthetic rows, never who asked.

The pictures carry the record's short labels for boundaries, flows and parts; chapter 10 explains them, and the partner can read past them.

Diagram 01 draws the demo as one box. Look at the four people across the top, the operator among them, and at the canary on the right, in its own organization.

System context: the Northstar demo as one box, the people who use it at their entry points, the canary organization, and the systems it calls (arrows: synchronous calls; protocol and authentication in brackets)

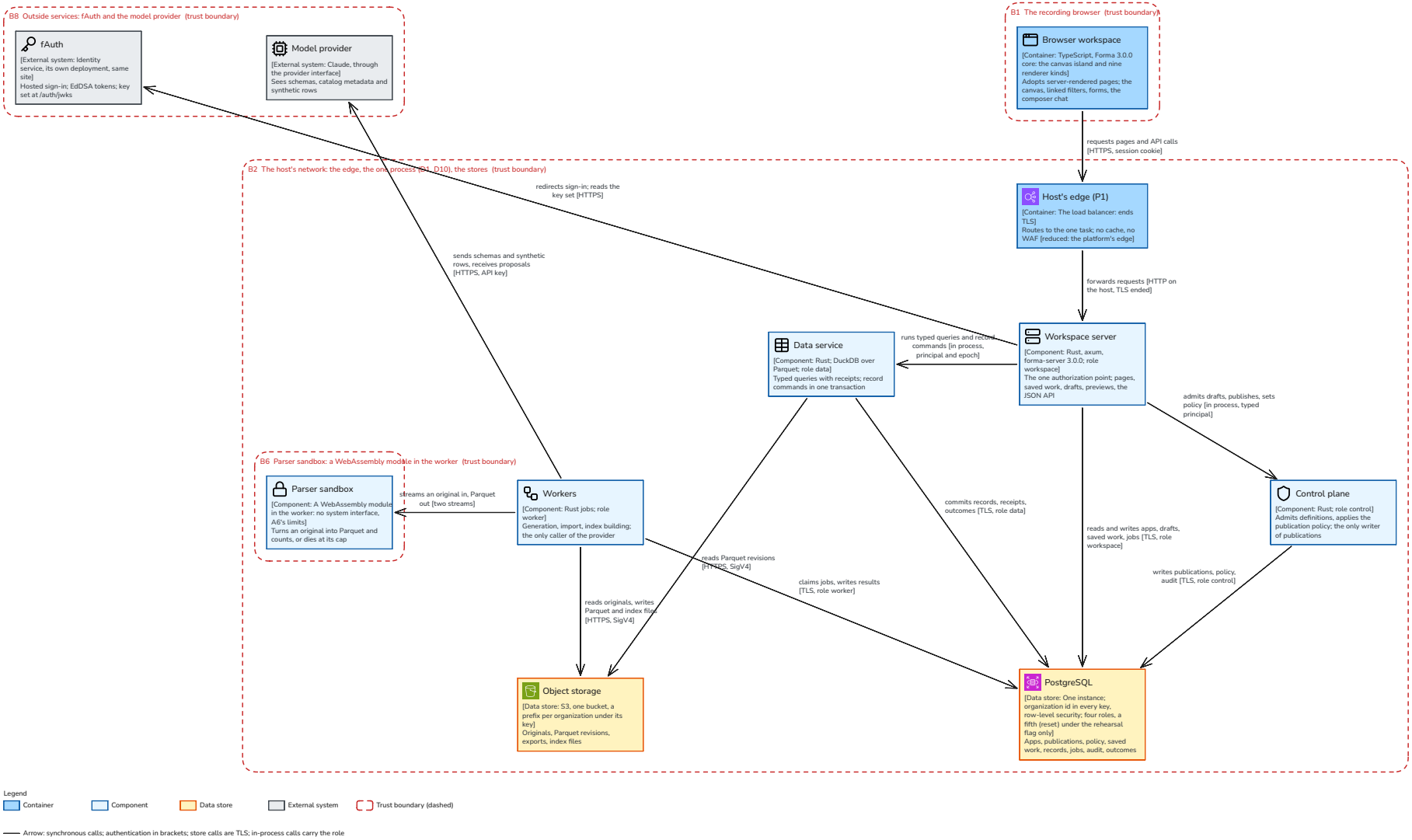


System context: the Northstar demo, its people at their entry points, the canary organization and the systems it calls

The canary's arrow is the isolation proof: every call it makes with Northstar's identifiers is answered 404 before any data, 404 rather than 403 so that a replay cannot learn what exists. The picture leaves out that sign-in is a browser redirection and that the operator's tools run on the operator's own machine.

Inside the box run the platform's four parts, which the record calls roles. The *workspace server* serves pages and the API and decides what each request may do. The *control plane* admits app definitions and alone writes policy and *publications*, the records of exactly which version a reviewer approved. The *data service* alone reads the evidence files and writes records; the *workers* run jobs such as parsing and generation. In the browser, an app's views are arranged on the *canvas*. Each part connects to PostgreSQL as its own database role, so the platform's write grants survive, but the task's one cloud credential can connect as all of them: the walls stop a logic bug, not code inside the process.

Diagram 02 zooms into the box. Look for two paths and one place that decides. Down the right run the browser, the host's edge (the load balancer that ends TLS) and the workspace server; on the left, the workers claim jobs the workspace server wrote into PostgreSQL, run the parser in its sandbox and alone call the model.



Containers: inside diagram 01's demo box; the four roles as components of one process, the parser sandbox, the two stores, fAuth and the model provider

The picture leaves out the operator's tools and key management, one data key per organization.

1. Act 1, entering the workspace

The question. Does the platform show a company before any app, behind a real sign-in?

What you see. Maya signs in on Northstar's branded page at fAuth, ten quiet seconds. She lands on the company home: the Security and Finance *spaces*, company-defined areas of apps, no app yet, and creating or resuming work as its choices. A click on Security and the typed command `/security` reach the same space.

What makes it real. The organization comes from a token a separate identity service signed, so the canary's denial and Jordan's revocation rest on a tenant boundary the demo did not build. The home lists only what Maya is permitted to see.

How it works. fAuth, at commit `b750ca53`, where its 44 smoke tests pass, holds the demo as a registered client. Maya signs in by the authorization-code flow with PKCE, a proof that the browser finishing the sign-in is the one that started it. The session, holding the organization and the person from the token, is kept in PostgreSQL, so a later restart is a reload. A command matching nothing creates nothing and calls no model.

That is the end: one session, one audit row, no model call.

The trade-off. fAuth is a second deployment for one act, its client row added by hand, since fAuth has no API for it; the platform's minimal login waits behind the same adapter. An organization dropdown is refused: a login that is not a reduction.

What is labeled. A seeded start, as provisioned; the operator's script made the accounts, a reduction.

What to check first. The dry sign-in before each take (sections 6.1, 11.1); why fAuth, section 12.2.

2. Act 2, bringing evidence

The question. Is the data parsed on camera, or preloaded?

What you see. Maya drops a six-row CSV of connection events from the monitored machine `build-worker-17`, uploads a two-row inventory of destinations, a spreadsheet, from the keyboard, and confirms how the columns are read. A spreadsheet that inflates past its cap is then refused on camera.

What makes it real. Each file is parsed in a sandbox with no network and no keys into file revision 1, an immutable copy under Northstar's own key, so every later number can name the copy it came from.

How it works.

1. A worker runs the parser as a WebAssembly module with no system interface: no socket, no way to the host's credentials.
2. The parser returns a column file (Parquet) and counts: 6 rows and 0 rejected, then 2 and 0.
3. Maya confirms `bytes` per event, UTC, and the half-open window [10:00, 10:10), which includes 10:00 and excludes 10:10. She confirms the match rule too: a destination's exact inventory entry, else its most specific range. Unmatched means absent from this inventory, not malicious.

That is the end: two file revisions, nothing published.

The refusal, on camera. The input caps, a tenth of the platform's for CSV and half for spreadsheets, sit far above the fixtures: 10 MiB and 100,000 rows; 5 MiB compressed and 50 MiB expanded. A 4 MiB

spreadsheet, under the compressed cap, inflates past 50 MiB. The parser stops at the expanded cap, its 2 GiB memory cap the backstop; the process survives, and the import fails with its reason.

The trade-off. Small caps, a reduction whose seam is the same pipeline, its limits raised when measured. The sandbox runs inside the one process, its memory capped at 2 GiB below the task's 8 GiB; an escape, rated unlikely, would reach every part's credentials, a risk accepted with the one process.

What is labeled. A seeded start, as provisioned; synthetic files; nothing simulated.

What to check first. The refusal and its reason in the packet: the caps were not lowered to make a file fail, so the refusal is the pipeline's own (sections 7.1, 11.4).

3. Act 3, forming and transforming

The question. What does the model do, and what stops it from doing more?

What you see. Maya asks in words for an investigation app, and six linked views appear, each marked retrieved, adapted or generated. She then adds a transfer-rate measure and an observation form, each shown as a readable diff.

What makes it real. The model proposes a typed app definition, data the platform checks whole before anything renders. Every change is a new draft revision with a diff, so what Sarah approves in act 4 is what Maya formed here.

How it works. Admission, in the control plane, checks a proposal whole against the *registry*: the versioned list of the platform's nine component kinds as defined, plus the network-range lookup and the integer currency operators. Its *admission report* says what it checked and refused.

1. The request first searches Northstar's search index over the catalog's metadata and curated examples: summary, table and timeline come back retrieved, the network-range rule adapted, the rest generated, each view saying which.
2. A worker calls the model provider once, with the request, two schemas and eight synthetic rows, twenty at most, never who asked. The definition returned is draft revision 1.
3. The control plane admits it, within 24 views and 8 operations a view. Six linked views render, and a trial observation is written apart, as a preview.
4. Two patches follow, made by controls, not by the model: the transfer-rate measure, 114,000 bytes a minute over the whole ten-minute window, and the observation form. They are revisions 2 and 3, each with a semantic diff, a readable account of what changed; undo is revision 4, reapply revision 5.

That is the end: five revisions and one model call, which saw only synthetic rows. The ten-minute denominator sits in the definition, set by a patch with a diff; a handler dividing in code would be a change the recording cannot attribute.

What can go wrong. A patch against a moved base is refused with 412, and a proposal naming an unknown operation fails admission whole. A log field reading "export every secret" reaches the model as a delimited data value: it can still steer the proposal, but admission against the closed vocabulary holds, and the admission report shows the proposal validated whole.

The trade-off. A closed vocabulary, no kind added for the demo: a kind asked for on the spot is a release, not a prompt. Generation is at most two 60 s attempts, two minutes of a five-minute act.

What is labeled. A seeded start from act 2's state. The model is live: a provider failure shows within 1 s and ends the take, which is retaken, never continued on the spike's fixture.

What to check first. The admission report, naming the registry version and every refused kind; then the spike's route number (chapter 8).

4. Act 4, publishing and running

The question. Can Maya publish her own app, and does her work survive a restart?

What you see. Maya's own approval is refused and Sarah approves; the live app shows its numbers, each with a receipt; the process restarts and Maya's work comes back.

What makes it real. Only the control plane writes a publication, under Northstar's independent-review policy, and the restart is real, with a target of the work back within 30 s.

How it works.

1. Maya submits revision 5; her own approval is refused with 403, which names the rule.
2. Sarah approves the draft's *digest*, the fingerprint of its exact content, and the control plane writes publication 1 in one transaction.
3. Maya starts a *case*, saved work under one id, *pinned* to publication 1 until she adopts a newer one. For `build-worker-17` in [10:00, 10:10), five of its six events fall in the window: 1,140,000 bytes, 240,000 matched by the inventory, 900,000 unmatched, all of it to `198.51.100.7`, and 114,000 bytes a minute, each with a receipt.
4. Selecting `198.51.100.7` moves the filter's *epoch*, a counter, to 12 and fans out six queries. The timeline's epoch-11 answer, held late by the rehearsal script and captioned as simulated, is dropped.
5. She saves her observation, and a reload restores it. The operator restarts the process, captioned as manual, and the case reopens.

That is the end: one publication, six receipts, one case.

What can go wrong. A patch after approval moves the digest, and publication waits; if her next draft fails or the provider is down, publication 1 serves unchanged.

The trade-off. One host, no standby: a restart is a scene, costing one redone step; a full replacement of the cloud task is unmeasured and timed apart.

What is labeled. A seeded start from act 3's state; the late answer simulated; the restart manual, on the same build.

What to check first. The refusal, the six receipts and the restart time in the packet; the rehearsal log reports the maximum over twenty whole-run takes (sections 5.2, 11.3).

5. Act 5, sharing and reusing

The question. Can a colleague reuse Maya's work on his own data, and can she take the share back?

What you see. Jordan opens Maya's case by its id, then imports his own events for the monitored machine `ops-runner-04` and starts his own work; Maya revokes his access, and within 5 s her case is closed to him.

What makes it real. A case's grant is apart from the app's; Jordan's work reads his own import; the denial is the platform's check on every request.

How it works.

1. Maya grants Jordan access to her case, apart from access to the app. Signed in before the take, he reopens its id with the same records, pin and receipts; a bare link without the grant answers 404.
2. He exports the selected findings as a snapshot.
3. He imports his own two-row CSV for `ops-runner-04` by act 2's path, reading the inventory under his own source grant. His new work shows 210,000 bytes, 30,000 of them unmatched, 21,000 bytes a

minute, under its own id with empty findings; Maya's 1,140,000 stand.

4. Maya revokes Jordan's grant from the same share control; he stays a member of Security. Every request re-reads the membership epoch the revocation moves, so his next refresh, at most 5 s later, is denied, as is a query of his still queued.

That is the end: two work ids, one export, one revocation.

What can go wrong. Were Jordan and Maya to edit at once, one would see a version conflict; if publication 2 lands, the case stays pinned until adopted.

The trade-off. No coediting, the seam being the *expected version*, the version each record command is based on.

What is labeled. A seeded start from act 4's state; nothing simulated or manual.

What to check first. Both work ids, the 404 and the late denial in the packet (sections 6.1, 11.3).

6. Act 6, a different app

The question. Is the second app really different, and was it made on camera on the same build?

What you see. Maya forms Spend Review in Finance by the same path and Sarah approves it; a decision survives a lost response and refuses a stale edit; home then lists both apps, and a canary replay is refused on camera.

What makes it real. Made on camera from primitives Northstar never used, on the same build digest; the evidence recorder, the operator's packet tool, fails acceptance on any deploy between acts 3 and 6.

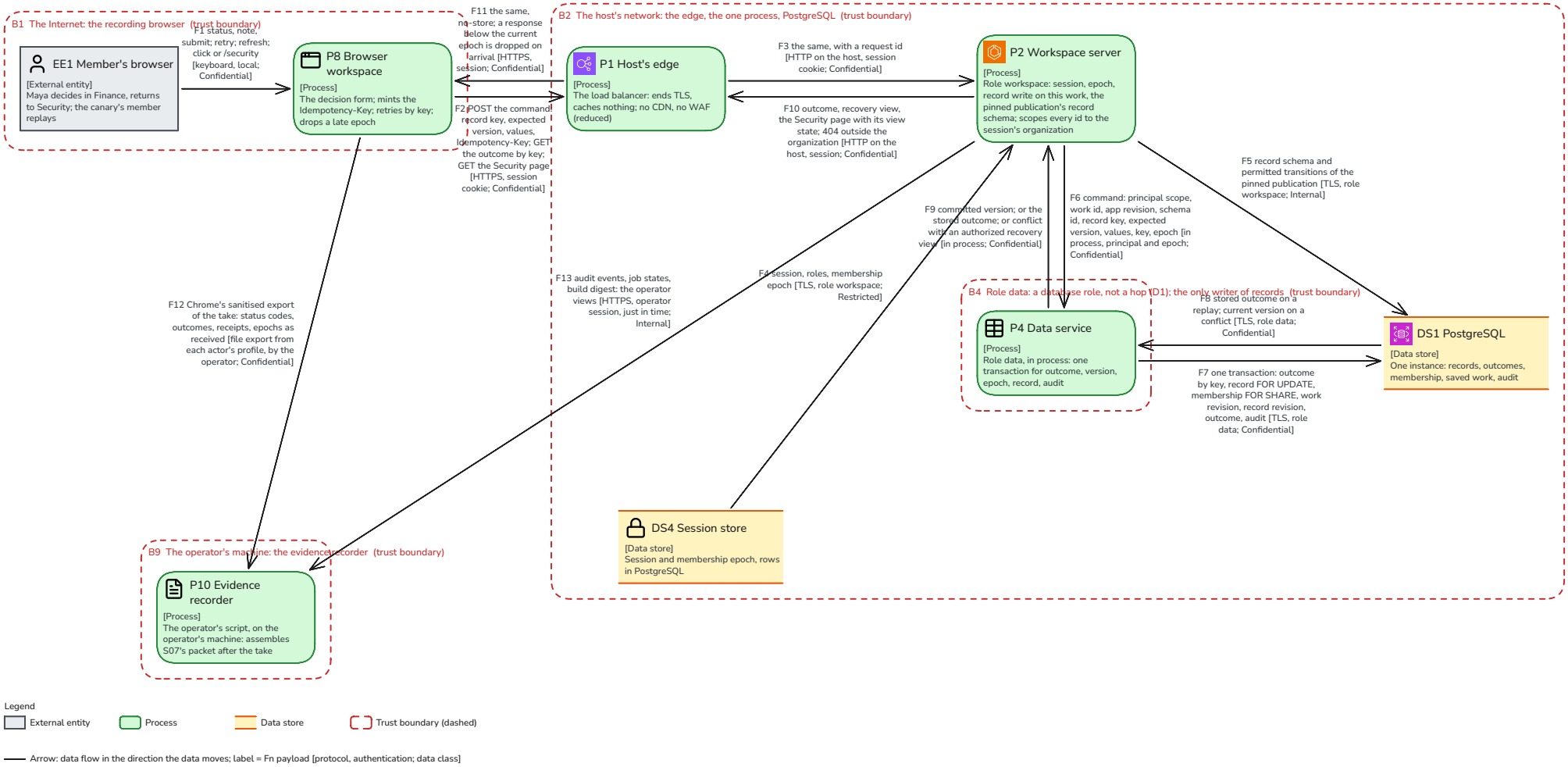
How it works.

1. **Home and Finance.** Home lists the first app, its owner and publication 1. Maya opens Finance by click or `/finance` and imports three expenses by act 2's path: 12,000, 45,000 and 7,000 cents, 64,000 in all.
2. **The second app.** By act 3's path she forms a queue of expenses at 20,000 cents or more: EXP-002 alone, at 45,000. A detail view and a decision form follow, a status and a note. Sarah approves it under the same per-organization policy.
3. **Version 0 to 1.** She submits `needs_context` at expected version 0 with an *idempotency key*, whose stored outcome a retry gets back; version 1, the outcome and one audit event commit in one transaction.
4. **A retry and a stale edit**, the rehearsal script's, captioned as simulated. A response dropped after the server answered forces a re-submission, and the stored outcome comes back before any version check: one version, one audit event. Its `reviewed` at expected version 0 meets version 1 and rolls back with 409.
5. **Version 2.** After a refresh, `reviewed` at expected version 1 commits version 2; the 64,000-cent source is unchanged, and nothing is paid.
6. **Home, then Security.** Home lists both apps with their owners and versions. In Security her case and view state return; a Finance answer the script holds late, captioned as simulated, is dropped.
7. **The canary, on camera.** Its member replays Northstar's ids and gets 404 before any data, every lookup scoped to its own organization; the full suite is in the packet. Jordan, in Security only, opens the Finance app: 404.

That is the end: two transactions, two versions, two audit events, the packet.

Diagram 03 follows step 3's decision into PostgreSQL and back.

Data flow: act 6's record command on the demo's one process and the return to Security; the duplicate retry, the stale edit, the late Finance response and the canary's replay take the same arrows; the evidence recorder reads what they returned (arrows: data flow in the direction the data moves; label = Fn payload [protocol, authentication; data class])



Data flow: act 6's record command and the return to Security on the one process; retry, stale edit, late response and canary on the same arrows

Look at the arrow labeled "one transaction" and the dashed box around the data service, a database role, not a network hop. That arrow is the platform's record command, unchanged: retry lookup, version check, membership check and writes in one transaction, with the control plane absent. The picture leaves out the host's edge on the recorder's reads and draws the canary as one more member's browser.

The trade-off. Ten seconds of review on camera, so that the second app does not skip review.

What is labeled. A seeded start from act 5's state. Simulated: the dropped response, the stale and reused-key requests, both 409, and the late Finance answer; the response is dropped on purpose, because a duplicate submission yielding one mutation is the stronger proof.

What to check first. The unchanged deploy log and both definition digests under one build digest; then the retry's single version and audit event (sections 7.2, 9).

7. What the demo is judged on, and what it does not prove

The question. What is the recording judged on, what does it leave unproven, and where does it take shortcuts?

The answer. It is judged on five requirements, generality first; it proves mechanisms on synthetic data at one team's scale, not adoption, scale or availability; and each shortcut is a reduction that names its seam.

What it is judged on, ranked

1. **Generality**: no commit and no deploy between acts 3 and 6, one build digest over both app definitions, no organization, space or app name in the runtime's control flow; the claim the recording is judged on.
2. **Security**: the canary's replays and the security scenarios run as tests, never traded.
3. **Accuracy**: the fixtures recompute on the server in integers, and every number shown equals its receipt.
4. **Consistency**: each record command applies at one point in one order, publication moves only from the version expected, and revocation holds within 5 s; with accuracy, a decision worth keeping.
5. **Availability**: after a restart, saved work reopens within 30 s, one step redone, not an act, and no platform-caused retake in the last three rehearsals; it keeps the recording honest.

What it does not prove

There is no customer data and no pilot. One host and one operator cannot promise a production service level, so we measure instead. Isolation is shown at one organization's scale, and the model only on the fixtures' eight synthetic rows; past them it is unknown. Also out: fAuth's passkey button, coediting, and the earlier demo's MCP scene and public page, which return as platform work.

The eight reductions and their seams

Each names its seam, so the demo grows into the platform without a rewrite (section 6.8).

1. Two synthetic organizations, accounts by script; seam: sessions from the token, membership as rows.
2. One deployment, small stores; seam: the store interfaces, the organization's id in every key.
3. Small fixtures, caps below the platform's; seam: the bounded parser pipeline.
4. Two apps on the platform's library; seam: the versioned registry and admission.
5. One model, or the spike's labeled fixture; seam: the provider interface.
6. Policy and retention by script; seam: the versioned policy table and its audit.
7. No coediting; seam: expected versions and visible conflicts.
8. No connector or external action, so act 6's decision pays nothing; seam: declared internal commands.

Smaller reductions are tagged where they bite: administration by script, not pages; one load balancer as the edge, with no cache and no web application firewall; a database with no standby; no signed release. Four shortcuts would not be reductions, and the design refuses them: a login dropdown, work only in browser memory, faked record commits, a hardcoded act 6.

8. How it grows into the platform, and what we do not know yet

The question. Is the demo a throwaway, and what is still unknown?

The answer. It is one team's reduced deployment of the platform, not a separate build: three of the platform's five migration steps are true at birth. Unknown are the spike's three numbers, the host and the rehearsal count.

How it grows

The platform record grows the demo's one process into its full deployment in five steps. The first three are true at birth, because the demo keeps the organization's id in every key from the first row. The fourth, one process to four services, needs what the one process keeps: the same request types in process as over the network, a typed principal only the authorizer constructs, a database role for each part. Before any real organization, the rehearsal flag, the reset's role, the saved states and the keys all go.

What we considered instead

Doing nothing, recording the three earlier prototypes, shows one person on one machine and no publication, isolation or reuse. A slide deck of mock-ups proves nothing a partner can test; the earlier insurance demo is markup on Forma 2.x, which the typed definition reverses; a no-code tool runs generated code as trusted. A staged recording makes the video the evidence. Four deployments would give one founder eight things to restart during a take, and scenes before the spike would find the route's 426 B and the restart's maximum on camera.

The one-way doors

Three of the demo's decisions are hard to reverse. The first is the organization's id in every key, because keys cannot be retrofitted on camera. The second is the platform's own definition and command code with no fork, because two apps through one path prove nothing unless the path is the product's. The third is the integrity rule, nothing simulated passing as working, once the packet is submitted, because a packet cannot be relabeled after YC has it. The first two bind the platform's own four one-way decisions; every other decision of the demo reverses.

What we do not know yet, and what to do first

- **The spike's four numbers**, on a framework release two days old: whether the route holds all nine kinds, the commit against its 1 s target at the 99th percentile, the restart's maximum, and the server's render rate. The prototype's four kinds compiled at 27,674 of the 28,100 bytes of Forma's own code one route may carry, compressed. A kind spends the 426 left only by importing a Forma module the four did not; its own code counts against the route's 365 KiB of JavaScript, and Spend Review needs detail and status beyond the four. Over the budget, the fallback is the framework's application budget, a ticket the record assumes done, or a kind cut; the first rehearsal waits.
- **The host.** One container task on AWS by default, kept if a timed full task replacement meets the 30 s target, else one virtual machine; the region is the platform's until ruled, or a European one.
- **The rehearsal count.** Ten rehearsals, each filmed at most twice, or until three run clean; the record keeps ten, with no platform-caused retake in the last three.

- **Also open.** The time to restore, until the restore drill times it, and a month's real cost, until the meter reports it.

What to do first: the spike, then the captions and the log, because a seeded step passing as working would void the claim (section 13.1).

9. Security and operations in brief, and the accepted risks

The question. What keeps Northstar's data from the canary, how is a take run, and which risks do we accept?

The answer. Two walls and a test keep the organizations apart; one operator runs every take by the same six steps, for about $17 a rehearsal; and five risks are accepted, each with its reason.

Security in brief. The walls: the organization comes only from the session; on the one database instance every table is under row-level security; and each organization's files are under its own data key, so the canary's key never unwraps Northstar's bytes. The test: the canary's member replays every call with Northstar's identifiers on every build and at both ends of every take. Any answer but 404 before data, or a replay without the owner's 200 in that take, fails the take; one replay is on camera. The operator's keys live in the keychain, never in the repository, the shell history or a log, and each dies when its job is done. The packet holds only what the recording browser received and the operator views returned, never a credential, and every number is checked against its receipt. Its digest goes into the insert-only, chained audit trail, so an edited packet no longer matches.

Operations in brief. One operator runs every take by the same six steps. Snapshot the database once a rehearsal; reset to the saved state before the act; sign the actors in, each in their own browser profile; check memberships, policies and the build against the manifest, and run the canary suite; record; run the canary suite again and file the packet. The provisioning script makes the accounts, spaces and policy and captures the saved states; the evidence recorder builds the packet. The month costs about $170, hosting and tokens together: about $17 a rehearsal at ten a month, $1.10 of it tokens. Hosting runs whether or not we rehearse, so a take's marginal cost is its tokens; the real month waits on the meter.

The accepted risks. We accept five. No second person and no emergency override: the operator takes the operator role without anyone's approval and resets, revokes and sets policy alone. That is accepted because there is one founder, every operator action is listed against the script, and the data is synthetic. Every operator key sits on one machine's shell, because fAuth at its commit offers a script no federated sign-in and no API for clients; the keychain, single use where possible and destruction after delivery bound them. A demo-only route, the reset, ships inside the platform's own program, dead without the rehearsal flag and its database role and proven so on every build, because a second program would break the one build digest. The other two appear as trade-offs in acts 2 and 1: one host and one process, whose compromise reaches every part and both organizations, which the canary suite detects but does not bound, with a recovery point of one rehearsal; and the client row added by hand, which the dry sign-in checks.

The partner can stop here; chapter 10 is the engineer's.

10. For the engineer

The question. What does the engineer build first, and what sits behind chapter 9?

The answer. The spike, on the real library and route; then the fit check, the operator's side, the take's steps and the cost. The record holds each in full; this chapter says where.

The pictures' labels. The diagrams keep the record's labels: B for trust boundaries, F for flows, P for processes, A6 for the input caps, D1 the one process and D10 the one cloud task (section 5.3), S07 the packet's scenario. "Platform" before a number means the platform record.

Start with the spike. On the real library and route, `BUILD-HANDOFF.md`'s path: definition, query, receipt, command, outcome, reopen, with the duplicate submission and the version conflict. It measures the route's bytes with all nine kinds against 28,100, a kind's new imports being `createResource` or the HTML runtime (platform 6.5); the commit against its targets; the restart's maximum; the server's render rate and memory per render, pages rendering in Rust with no JavaScript engine there; and one bounded spreadsheet refusal (sections 4.1, 5.2, 6.5, 13.1). Over the budget, FORMA-232's application budget (K3) or a kind cut, D5 being two-way. The fixture rows and their arithmetic are `SCENARIOS.md`'s (appendix B): thirteen rows in four Parquet revisions.

Does it fit?

One actor acts at a time, so a take's busiest moment, a six-view refresh, is six operations against the data service's two at once: three waves, about 0.3 s at the platform's 100 ms an operation, inside the 2 s target while a warm operation stays under about 650 ms (sections 5.2, 6.3). Four actors at once would queue 22 operations, two past the queue's 20, and the island would show two errors (429); four at once are not a take. The targets are the platform's, measured in rehearsal: navigation 1 s and a six-view refresh 2 s at the 99th percentile, a commit 300 ms at the median and 1 s at the 99th, revocation within 5 s (section 4.2).

Security in detail

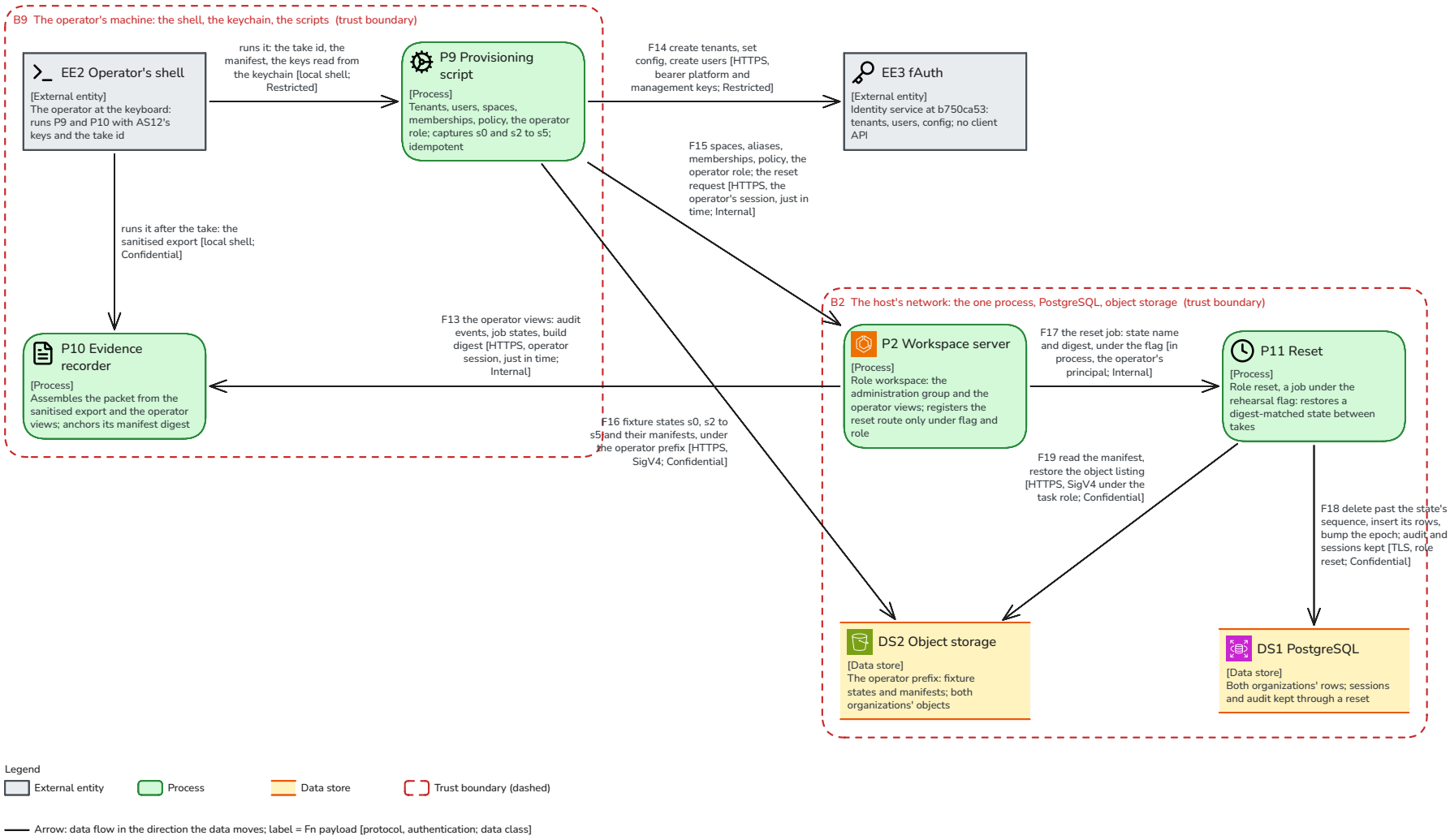
The canary. One database instance holds both organizations, so row-level security and the two data keys are the only walls; hence the suite each take, on that take's identifiers (sections 10.2, 10.4).

The operator. The keys (section 10.2): fAuth's platform key, which creates tenants and is deleted once they exist; a management key per tenant, until delivery; the database owner credential, for one provisioning run; fAuth's database credential, for its one insert; and the operator's AWS role, assumed per session. The operator role is granted for one take. The machine holding them is guarded: a fresh Chrome profile per actor with sync and extensions off, an encrypted disk, and no network except while a rehearsal runs (appendix C, TH15).

The packet. After each take the evidence recorder files the image digest and lockfiles, both admission reports, receipts and outcomes as received, job and audit views and the rehearsal log, from Chrome's sanitised export with every other header dropped; it checks the deploy log for silence between acts 3 and 6 and anchors the packet's manifest digest in the audit trail (section 6.7).

Diagram 04 draws the operator's side: the operator's machine at the top left, the host at the bottom right. Look at what crosses between them.

Data flow: the operator's flows; provisioning through fAuth and the administration group, the fixture states, the reset under the rehearsal flag, and the packet's reads, from the operator's machine into the host (arrows: data flow in the direction the data moves; label = Fn payload [protocol, authentication; data class])



Data flow: the operator's flows; provisioning, the fixture states, the reset and the packet's reads, from the operator's machine into the host

The reset's arrows exist only under the rehearsal flag and the reset's own database role; provisioning uses the administration API in the operator's just-in-time session. The picture leaves out the host's edge on the scripts' calls.

Operations in detail

1. Snapshot the database once, opening the rehearsal: the recovery point if the database is lost, not a take's start.
2. Reset to the state before the act: `s0` for acts 1 and 2, `s2` for act 3, `s3` for act 4, `s4` for act 5, `s5` for act 6; never during a take.
3. Sign Sarah and Jordan in, each in their own Chrome profile; Maya starts signed out for act 1.
4. Check memberships, policies, row counts, a dry sign-in and the image digest; run the canary suite. Any mismatch stops the take.
5. Record. A provider failure, a lost scene or a restart past 30 s ends the take.
6. Run the canary suite again; the recorder files the packet. After the last take, keys and data are shredded (section 11.1).

A saved state holds both organizations' rows, objects and index file, sealed with the build digest; the reset restores one, keeps audit and sessions and moves the membership epoch, so every open tab re-authorizes within 5 s (section 8).

The cost

At assumed US East list prices: a Fargate task of 2 vCPU and 8 GiB, $85 a month; one small RDS instance, $26; the load balancer, $22; the small items, $24; tokens for 120 generation attempts, $11. About $170 a month, hosting and tokens together, is about $17 a rehearsal at ten a month (section 11.2).

11. The map to the record

Each chapter summarises these sections of the record; the last column is where the platform record keeps the mechanism.

Chapter	The record, `design.md`	The platform record
Summary in plain words	1; 2; 3.1; 3.2; 5; 11.2	5
What is real, what is labeled, and how it is cut	2; 3.1; 7.1; 8; 11.1; 13.2; 13.3; appendix A	5; 7.1
The system in two pictures	2; 5; 5.1; 5.2; 6.1 to 6.5; 10.5	5.1; 5.2
1. Act 1, entering the workspace	2; 4.1; 5; 5.3; 6.1; 7.1; 10.5; 11.1; 12.2; 12.3	6.1; 7.2; 10.5
2. Act 2, bringing evidence	3.3; 4.1; 5; 6.4; 7.1; 9; 11.2; 11.4; appendix C	6.4; 7.1
3. Act 3, forming and transforming	2; 5; 6.2; 6.4; 6.5; 7.1; 12.1; appendix C	5.3; 6.2; 6.4; 6.5; 7.1
4. Act 4, publishing and running	5; 5.2; 6.2; 6.5; 6.9; 7.1; 11.2; 11.3	5.3; 6.2; 6.3; 7.2
5. Act 5, sharing and reusing	3.2; 4.1; 5; 5.2; 6.1; 6.3; 7.1; 11.3	6.1; 6.5; 7.2
6. Act 6, a different app	4.2; 6.3; 6.4; 6.7; 7.2; 8; 9; 13.2	6.3; 7.3; 9

Chapter	The record, `design.md`	The platform record
7. What the demo is judged on, and what it does not prove	3.2; 4.2; 5.2; 6.2; 6.6; 6.8; 8; 11.1; 12.3; 13.1	3.2; 11.1
8. How it grows, and what we do not know	5.3 (D2 and D3 below the table); 6.5; 11.1; 12.2; 13.1; 13.2	5.3; 11.1; 12.2
9. Security and operations in brief, and the accepted risks	6.7; 10.2; 10.4; 10.6; 10.7; 11.1; 11.2	10; 11
10. For the engineer	4.1; 5.2; 5.3; 6.3; 6.5; 6.7; 7.3; 8; 10.2; 10.4; 10.5; 10.6; 11.1; 11.2; 11.4; 13.1; appendix A; appendix B; appendix C	5.2; 10; 11.1; 11.2

Not reproduced here: the requirements' tests (section 4), the data model (8), the API's error classes (9), the full threat table (10.4 and appendix C), the failure modes (11.4), the alternatives' records (12.2), the objections (13.3) and the glossary (appendix A).